

Introduction to Reinforcement Learning

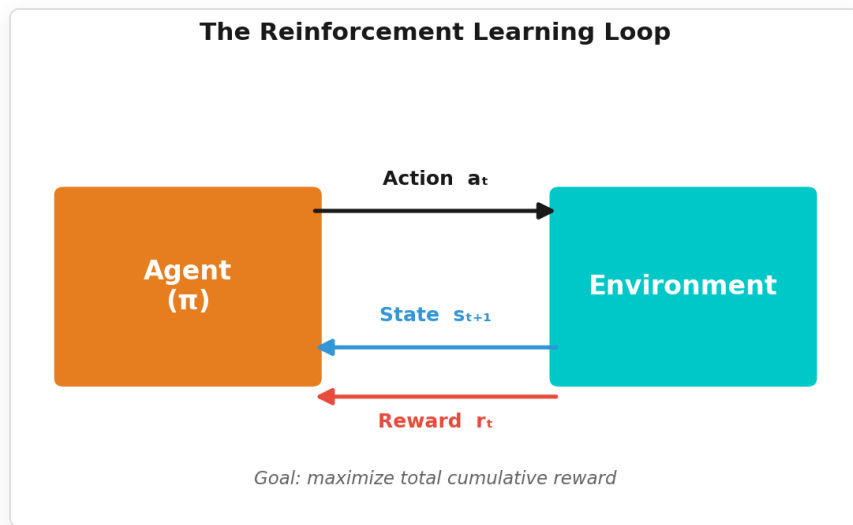
Applied Machine Learning — Session 4, Chapter 1

The Third Paradigm

Paradigm	Signal	Example
Supervised	Labels (X, y)	Predict house price
Unsupervised	None	Find customer groups
Reinforcement	Rewards from trial & error	Train a game agent

What Is Reinforcement Learning?

An agent learns to make decisions by interacting with an environment.



Goal: Maximize total cumulative reward over time.

Everyday Analogies

Training a dog:

- Give treats for good behavior → positive reward
- Ignore bad behavior → no reward
- Dog learns: some actions lead to treats

Learning to ride a bike:

- Falling → negative reward
- Staying balanced → positive reward
- You adapt through experience alone

Playing a video game:

- Score = reward. Controls = actions. Game = environment.

Core Components

Term	Meaning	Example (grid world)
State (s)	Current situation	Position (row, col)
Action (a)	What the agent can do	Move North/South/East/West
Reward $\mathcal{R}$	Feedback signal	+1 at goal, -1 in hole
Policy (π)	Strategy: $s \rightarrow a$	"If at (3,2), go North"
Episode	One full run	Start $\rightarrow$ goal (or failure)

Markov Decision Process (MDP)

Formal framework: (S, A, P, R, γ)

- S = states, A = actions
- $P(s'|s, a)$ = transition probability
- $R(s, a)$ = reward function
- γ = discount factor (0 to 1)

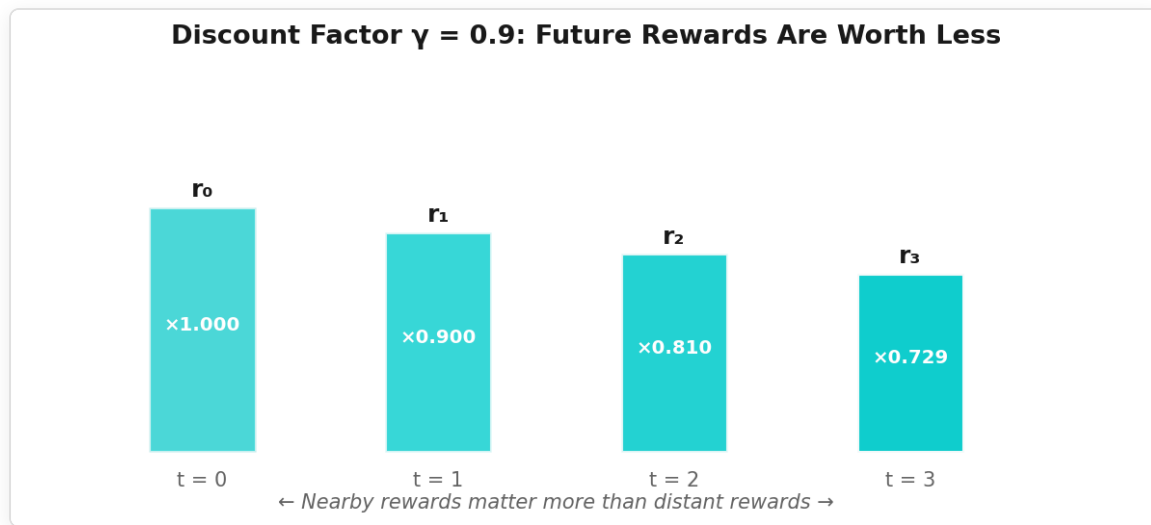
The Markov Property:

"The future depends only on the current state — not on history."

→ You only need to know *where you are*, not *how you got there*.

The Discount Factor γ

How much do we value future rewards?



- $\gamma = 0$: Only care about immediate reward (greedy, shortsighted)
- $\gamma = 1$: Care equally about all future rewards
- $\gamma = 0.9$: Typical — future rewards worth 90% of current

Exploration vs Exploitation

The fundamental dilemma:

- **Exploitation:** Do what you know works best
- **Exploration:** Try something new — might be better!

Restaurant analogy:

Exploit → go to your favorite restaurant (safe)

Explore → try somewhere new (risky, but maybe amazing!)

ϵ -Greedy: The Solution

With probability $\epsilon \rightarrow$ explore (random action)

With probability $1-\epsilon \rightarrow$ exploit (best known action)

```
if random() < epsilon:
    action = random_action()      # explore
else:
    action = argmax(Q[state])     # exploit
```

Epsilon decay: Start high (0.9), reduce over time (end at 0.01) $\rightarrow$ "Learn first, then use what you learned."

Epsilon Decay Schedule

```
 $\epsilon = \max(\epsilon \times \text{decay\_rate}, \text{min\_epsilon})$ 
```

Phase	Episodes	ϵ range	Behavior
Early	0–200	0.9 → 0.5	Mostly exploring
Mid	200–600	0.5 → 0.1	Balancing
Late	600–1000	0.1 → 0.01	Mostly exploiting

Why decay?

→ At start: know nothing → explore everything

→ At end: know the environment → use that knowledge

Too fast decay = stops exploring before learning enough

Too slow decay = wastes episodes on random actions

Value Functions

$Q(s, a)$: Expected total reward after taking action a in state s

Q-Table:

	Left	Down	Right	Up
State 0:	0.1	0.3	0.5	0.0
State 1:	0.0	0.8	0.2	0.1
...				

Optimal policy: Always take the action with the highest Q-value!

```
optimal_action(s) = argmax_a Q(s, a)
```

Deterministic vs Stochastic Environments

Our grid world is **deterministic** — actions always go as planned.

Real environments are often **stochastic** — the agent may slip!

Deterministic: action=Right → always move Right

Stochastic: action=Right → 70% Right, 15% Up, 15% Down

Our grid world in the examples is deterministic.

*FrozenLake (Ch11) adds **stochasticity** — the agent slips!*

The Q-Learning algorithm handles both.

Real-World Applications

Domain	Agent	Environment	Reward
Games	AI player	Game board	Win/lose/score
Robotics	Robot	Physical world	Task complete
Finance	Trading bot	Market	Profit
Healthcare	Treatment planner	Patient	Health outcome
Traffic	Signal controller	Road network	Reduced delays

Famous Milestones

- **2013:** DeepMind — superhuman Atari games from pixels
- **2016:** AlphaGo — defeats world Go champion
- **2019:** AlphaStar — defeats top StarCraft II players
- **2022:** AlphaCode — competitive programming

Now: Live Example!

→ Open `02-examples/ch10_rl_intro_examples.ipynb`

We will:

1. Build a grid world environment
2. Watch a random agent bumble around
3. Compare it to an optimal policy
4. See the learning problem clearly

Key Takeaways

- RL = learning by trial and error with rewards
- Agent $\leftrightarrow$ Environment loop: state $\rightarrow$ action $\rightarrow$ reward $\rightarrow$ next state
- MDP: formal framework (S, A, P, R, γ)
- Exploration vs Exploitation: the core dilemma
- Q-values: estimate of long-term value of each (state, action) pair

Next: Chapter 11

Basic RL Algorithms

"Now let's teach the agent to learn – with Q-Learning."